

Module 2: Lesson 3

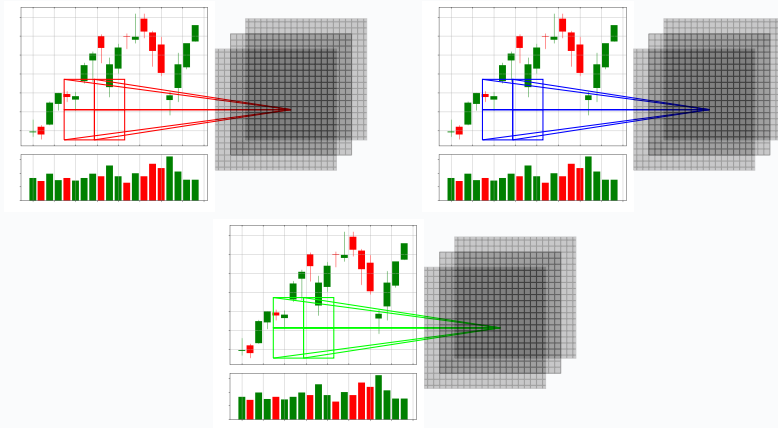
Convolutional Neural Networks: Towards Modern CNNs



Outline

- ▶ Multiple input and output channels
- ▶ Pooling
- ▶ Batch Normalization

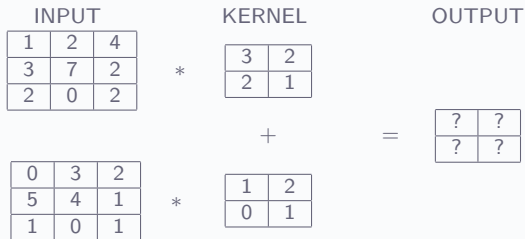
Multiple input channels



Multiple input channels

When the input data contains multiple channels, we need to construct a convolution kernel with the same number of input channels as the input data.

That is, we need a kernel that contains a tensor of shape $\text{kernel height} \times \text{kernel width}$ for every input channel.



Multiple input channels

To obtain the output channel we just add up the results of the convolution for each separate input channel.

INPUT				KERNEL			OUTPUT	
1	2	4		3	2			
3	7	2	*	2	1			
2	0	2						
						+		
0	3	2		1	2			
5	4	1	*	0	1			
1	0	1				=		
							30 ?	
							? ?	

Multiple input channels

To obtain the output channel we just add up the results of the convolution for each separate input channel.

INPUT				KERNEL			OUTPUT	
1	2	4		3	2			
3	7	2	*	2	1			
2	0	2						
					+			
0	3	2		1	2	=	30 38	
5	4	1	*	0	1		40 34	
1	0	1						

Remember that in practice we also need to add a bias term associated with each kernel.

Multiple output channels

When training CNNs, it turns out to be essential to have multiple channels and increase the channel dimension at each layer.

The intuition is that each channel will detect a different set of features when training the network.

For each output channel, we create a kernel tensor of shape kernel height $\times$ kernel width $\times$ input channels.

INPUT		KERNEL Channel 1		OUTPUT Channel 1													
<table><tr><td>1</td><td>2</td><td>4</td></tr><tr><td>3</td><td>7</td><td>2</td></tr><tr><td>2</td><td>0</td><td>2</td></tr></table>	1	2	4	3	7	2	2	0	2	*	<table><tr><td>3</td><td>2</td></tr><tr><td>2</td><td>1</td></tr></table>	3	2	2	1		
1	2	4															
3	7	2															
2	0	2															
3	2																
2	1																
		+															
		=	<table><tr><td>30</td><td>38</td></tr><tr><td>40</td><td>34</td></tr></table>	30	38	40	34										
30	38																
40	34																
<table><tr><td>0</td><td>3</td><td>2</td></tr><tr><td>5</td><td>4</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	0	3	2	5	4	1	1	0	1	*	<table><tr><td>1</td><td>2</td></tr><tr><td>0</td><td>1</td></tr></table>	1	2	0	1		
0	3	2															
5	4	1															
1	0	1															
1	2																
0	1																

INPUT		KERNEL Channel 2		OUTPUT Channel 2													
<table><tr><td>1</td><td>2</td><td>4</td></tr><tr><td>3</td><td>7</td><td>2</td></tr><tr><td>2</td><td>0</td><td>2</td></tr></table>	1	2	4	3	7	2	2	0	2	*	<table><tr><td>0</td><td>2</td></tr><tr><td>1</td><td>1</td></tr></table>	0	2	1	1		
1	2	4															
3	7	2															
2	0	2															
0	2																
1	1																
		+															
		=	<table><tr><td>27</td><td>30</td></tr><tr><td>27</td><td>11</td></tr></table>	27	30	27	11										
27	30																
27	11																
<table><tr><td>0</td><td>3</td><td>2</td></tr><tr><td>5</td><td>4</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	0	3	2	5	4	1	1	0	1	*	<table><tr><td>1</td><td>1</td></tr><tr><td>2</td><td>0</td></tr></table>	1	1	2	0		
0	3	2															
5	4	1															
1	0	1															
1	1																
2	0																

The 1×1 convolutional layer

1×1 convolutions are sometimes included in the designs of complex deep networks. These convolutions imply a kernel of dimension 1×1 .

In a single input and output channel framework, the 1×1 convolution only maps the input image to an identical output layer.

1×1 convolutions only work through the channel dimension. This type of convolution leads to output layers with the same height and width as the input.

1×1 convolutions generate linear combinations of elements at the same position across the input channels. However, layers are usually followed by non-linear activations, so that these type of convolutions are not subsumed into other convolutions with kernels of higher dimension.

This type of convolution loses the ability to recognize patterns of interactions among adjacent pixels, but aims at detecting the relevance of specific pixels.

Pooling

In most cases, our ultimate objective about image recognition is to obtain a prediction about the content of a given image. For instance, does a particular candlestick pattern predict a positive or a negative return?

For this, the final layer in a CNN should be sensitive to the entire input image.

Another issue with CNNs is that the output may be sensitive to minor changes in the location of features within images.

Pooling layers serve the dual purpose of downsampling representations while mitigating the sensitivity of convolutional layers to location.

Pooling operators work as convolutions by applying an operation with a fixed window that travels over the input image but, different from convolutions, pooling layers contain no parameters. Besides, pooling leaves unchanged the channel dimension.

The most common Pooling layer is max-pooling (the alternative option is average-pooling):

INPUT		KERNEL		OUTPUT															
<table border="1"><tr><td>1</td><td>2</td><td>5</td></tr><tr><td>3</td><td>4</td><td>2</td></tr><tr><td>6</td><td>1</td><td>3</td></tr></table>	1	2	5	3	4	2	6	1	3		<table border="1"><tr><td colspan="2">2×2 Pooling Layer</td></tr></table>	2×2 Pooling Layer		=	<table border="1"><tr><td>4</td><td>5</td></tr><tr><td>6</td><td>4</td></tr></table>	4	5	6	4
1	2	5																	
3	4	2																	
6	1	3																	
2×2 Pooling Layer																			
4	5																		
6	4																		

Pooling with multiple channels

When processing input layers with multiple channels, the pooling layer treats each input channel separately.

Thus, the number of output channels in a pooling layer is the same as the number of input channels.

Bear in mind also that we can change the pooling window as well as include padding and stride to any pooling layer.



Batch normalization

Making neural networks converge is one of the biggest challenges in the design of architectures: **Batch Normalization** is a popular technique that consistently accelerates convergence.

Batch normalization can be understood in terms of the variable rescaling that we use in data pre-processing.

- ▶ In each training iteration, we first normalize the inputs by subtracting their mean and their standard deviation within the current minibatch.
- ▶ Next, we apply a linear transformation (slope + intercept).

With batch normalization, the choice of batch size becomes even more significant than in its absence

- ▶ Batch normalization works best for moderate minibatches (50 to 100 instances).

Batch normalization adds sufficient noise to training to improve the convergence speed and reduce overfitting, as in dropout.

This tool also reduces the variation in the values of the hidden units that may hamper convergence.

Batch normalization: Details

Denote by $\mathbf{x} \in \mathcal{B}$ an input to Batch Normalization BN from minibatch $\mathcal{B}$. $\mathbf{x}$ has a dimension equal to the size (number of instances) of the minibatch.

Batch normalization transforms $\mathbf{x}$ according to the following expression:

$$BN(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta$$

$$\hat{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x}$$

$$\hat{\sigma}_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2$$

where $\hat{\mu}_{\mathcal{B}}$ is the sample mean and $\hat{\sigma}_{\mathcal{B}}$ the sample standard deviation of the minibatch $\mathcal{B}$, and $\odot$ refers to element-by-element multiplication.

γ and β are trainable scale and shift parameters with the same shape as $\mathbf{x}$.

Batch normalization: Details

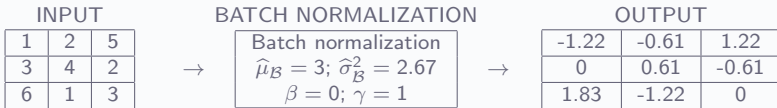
In fully-connected layers of MLPs, if we have a weight parameter $\mathbf{W}$, bias $\mathbf{b}$, and activation function ϕ , an output layer $\mathbf{h}$ with batch normalization is computed as follows:

$$\mathbf{h} = \phi(BN(\mathbf{W}\mathbf{x} + \mathbf{b}))$$

where the mean and variance are computed over the “feature” dimension.

In CNNs, we apply batch normalization on a per-channel basis across all locations.

- ▶ We compute the mean and variance of an input channel across all locations and examples.
- ▶ We apply the same mean and variance to normalize within a given channel so that each channel has one scale parameter and one shift parameter.



Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ The role of input and output channels in convolutions.
- ▶ The pooling operator.
- ▶ Batch normalization.

⇒ **References for this lesson:**

Chapters 7 & 8 in [Zhang, Aston, et al. "Dive into deep learning." \(2022\).](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement Deep CNNs using Tensorflow-Keras in our candlesticks application.